





In [1]:

```
import warnings; warnings.filterwarnings("ignore")
import numpy as np, pandas as pd, time
from collections import OrderedDict
from sklearn.base import BaseEstimator, TransformerMixin, clone
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.model_selection import (StratifiedKFold, RepeatedStratifiedKFold,
                                     GridSearchCV)
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import (RandomForestClassifier, ExtraTreesClassifier,
                              GradientBoostingClassifier, VotingClassifier)
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import NearestNeighbors
from sklearn.calibration import calibration_curve
from sklearn.metrics import (f1_score, roc_auc_score, balanced_accuracy_score,
                             matthews_corrcoef, confusion_matrix, classification_report, roc_curve, auc,
                             precision_recall_curve, average_precision_score, brier_score_loss)
from scipy.stats import t as tdist, chi2, friedmanchisquare, mannwhitneyu
import matplotlib.pyplot as plt, seaborn as sns

try:    from xgboost import XGBClassifier; XGB = True
except Exception: XGB = False
try:    import shap; SHAP = True
except Exception: SHAP = False

RS = 42
N_OUTER_SPLITS, N_OUTER_REPEATS = 5, 10
N_INNER_SPLITS = 5
TUNE_THRESHOLD_DEFAULT = False
RUN_FULL_MODEL_COMPARISON = True

np.random.seed(RS)
plt.rcParams["figure.dpi"] = 110; sns.set_style("whitegrid")

DATA = "Plantar Thermogram Database.xlsx"
ANG = ["General", "LCA", "LPA", "MCA", "MPA", "TCI"]; REG = ANG[1:]
Rc = [f"R_{a}" for a in ANG]; Lc = [f"L_{a}" for a in ANG]
CLIN = ["Age", "IMC"]; BIO = ["TAI", "RHHI", "PTVI", "ASI", "TSS"]
EPS = 1e-8
AGE_LO, AGE_HI = 23, 52
def firth_logit(X, yv, max_iter=200, tol=1e-8):
    X = np.atleast_2d(np.asarray(X, float))
    if X.shape[0] != len(yv): X = X.T
    yv = np.asarray(yv, float)
    Xd = np.column_stack([np.ones(len(yv)), X]) if X.size else np.ones((len(yv), 1))
    beta = np.zeros(Xd.shape[1])
    for _ in range(max_iter):
        pr = np.clip(1/(1+np.exp(-(Xd@beta))), 1e-10, 1-1e-10); W = pr*(1-pr)
        inv = np.linalg.pinv(Xd.T @ (Xd*W[:,None]))
        h = np.sum((Xd@inv)*Xd, axis=1)*W
        step = inv @ (Xd.T @ (yv - pr + h*(0.5-pr))); nb = beta + step
        for _ in range(25):
            pt = np.clip(1/(1+np.exp(-(Xd@nb))), 1e-10, 1-1e-10)
            s, ld = np.linalg.slogdet(Xd.T @ (Xd*(pt*(1-pt))[:,None]))
            if np.isfinite(ld) and s > 0: break
        step /= 2; nb = beta + step
```

```

        if np.max(np.abs(nb-beta)) < tol: beta = nb; break
        beta = nb
    pr = np.clip(1/(1+np.exp(-(Xd@beta))), 1e-10, 1-1e-10); W = pr*(1-pr)
    se = np.sqrt(np.clip(np.diag(np.linalg.pinv(Xd.T @ (Xd*W[:,None]))), 0, None))
    ll = np.sum(yv*np.log(pr)+(1-yv)*np.log(1-pr))
    _, ld = np.linalg.slogdet(Xd.T @ (Xd*W[:,None]))
    return beta, se, ll + 0.5*ld

def firth_plrt(Xf, yv, drop):
    llf = firth_logit(Xf, yv)[2]
    keep = [k for k in range(Xf.shape[1]) if k not in np.atleast_1d(drop)]
    llr = firth_logit(Xf[:, keep] if keep else np.empty((len(yv),0)), yv)[2]
    st = max(2*(llf-llr), 0.0)
    return st, chi2.sf(st, len(np.atleast_1d(drop)))

def holm(pairs):
    srt = sorted(pairs, key=lambda t:(np.inf if t[1]!=t[1] else t[1]))
    m=len(srt); prev=0.0; out={}
    for i,(k,p) in enumerate(srt):
        adj = np.nan if p!=p else min(1.0, max(prev,(m-i)*p))
        if adj==adj: prev=adj
        out[k]=adj
    return out

def ci95(s):
    s=np.asarray(s); m=s.mean(); sem=s.std(ddof=1)/np.sqrt(len(s))
    return m, tdist.interval(0.95, len(s)-1, loc=m, scale=sem)

def nadeau_bengio(d, n_tr, n_te):
    d=np.asarray(d,float); n=len(d)
    if d.std(ddof=1)==0: return np.nan, np.nan
    ts=d.mean()/np.sqrt(d.var(ddof=1)*(1.0/n + n_te/n_tr))
    return ts, 2*(1-tdist.cdf(abs(ts), df=n-1))

def cliffs_delta(x,z):
    x,z=np.asarray(x),np.asarray(z)
    gt=sum(np.sum(xi>z) for xi in x); lt=sum(np.sum(xi<z) for xi in x)
    return (gt-lt)/(len(x)*len(z))

def dlab(d):
    a=abs(d)
    return "negligible" if a<.147 else "small" if a<.33 else "medium" if a<.47

def ece(yt, pp, n_bins=10):
    conf=np.maximum(pp,1-pp); pc=(pp>=.5).astype(int); corr=(pc==yt).astype(float)
    bins=np.linspace(0,1,n_bins+1); e=0.0
    for lo,hi in zip(bins[:-1],bins[1:]):
        m=(conf>lo)&(conf<=hi)
        if m.sum(): e += (m.sum()/len(yt))*abs(corr[m].mean()-conf[m].mean())
    return e

# %% ===== DATA + REPRESENTATION =====
def _sheet(s, lab):
    d = pd.read_excel(DATA, sheet_name=s, header=None).iloc[2:].copy()
    d.columns = ["Subject", "Gender", "Age", "Weight", "Height", "IMC"]+Rc+Lc+["x18"]
    d = d[d["Subject"].astype(str).str.match(r"(CG|DM)").copy()]; d["group"]=lab

df = pd.concat([_sheet("Control Group","CG"), _sheet("DM Group","DM")], ignore_index=True)
for c in ["Age", "Weight", "Height", "IMC"]+Rc+Lc: df[c]=pd.to_numeric(df[c],errors='coerce')
df = df.reset_index(drop=True)
y = LabelEncoder().fit_transform(df["group"]); classes=["CG", "DM"]

```

```

def biomarkers(X):
    Rv={a:X[f"R_{a}"].values for a in ANG}; Lv={a:X[f"L_{a}"].values for a in
    Rm=np.column_stack([Rv[a] for a in REG]); Lm=np.column_stack([Lv[a] for a
    b=OrderedDict()
    b["TAI"]=np.mean([np.abs(Rv[a]-Lv[a]) for a in REG],axis=0)
    b["RHHI"]=(Rm.std(1)+Lm.std(1))/2
    allm=np.column_stack([Rm,Lm]); b["PTVI"]=allm.max(1)-allm.min(1)
    Rr=Rm-Rm.mean(1,keepdims=True); Ll=Lm-Lm.mean(1,keepdims=True)
    b["ASI"]=(np.abs(Rr).mean(1)+np.abs(Ll).mean(1))/2
    den=(Rm.mean(1)+Lm.mean(1))/2+EPS
    b["TSS"]=1-(np.mean([np.abs(Rv[a]-Lv[a]) for a in REG],axis=0)/den)
    return b

class Features(BaseEstimator, TransformerMixin):
    def __init__(self, blocks=("thermal",)): self.blocks = blocks
    def fit(self, X, y=None):
        self.imp_ = SimpleImputer(strategy="median").fit(X[CLIN]); return self
    def transform(self, X):
        cols,names=[],[]
        if "thermal" in self.blocks:
            for a in ANG:
                r,l = X[f"R_{a}"].values, X[f"L_{a}"].values
                cols.append(np.abs(r-l)); names.append(f"asym_{a}")
                cols.append((r+l)/2); names.append(f"mean_{a}")
        if "biomarkers" in self.blocks:
            for k,v in biomarkers(X).items(): cols.append(v); names.append(k)
        if "clinical" in self.blocks:
            cl=self.imp_.transform(X[CLIN])
            for i,c in enumerate(CLIN): cols.append(cl[:,i]); names.append(c)
        self.feature_names_=names
        return np.column_stack(cols)

def make_pipe(model, blocks):
    return Pipeline([("feat",Features(blocks)),("scale",StandardScaler()),("cl

B = biomarkers(df)
A = pd.DataFrame({k:B[k] for k in BIO})
A["Age"]=df["Age"].values
A["BMI"]=SimpleImputer(strategy="median").fit_transform(df[["IMC"]]).ravel()
A["DM"]=y
Z = A.copy()
for c in BIO+["Age","BMI"]: Z[c]=(A[c]-A[c].mean())/A[c].std(ddof=0)
yv = Z["DM"].values

ENSEMBLE_POOL = OrderedDict([
    ("rf", lambda: RandomForestClassifier(n_estimators=150, class_weight="bala
    ("gb", lambda: GradientBoostingClassifier(random_state=RS)),
])
if XGB:
    ENSEMBLE_POOL["xgb"] = lambda: XGBClassifier(n_estimators=150, max_depth=3
                                                random_state=RS, eval_metric=

CANDIDATE_SUBSETS = [ ["rf","gb"], ["rf","gb","xgb"] ]
CANDIDATE_WEIGHTS = [None, [2,1], [1,2], [2,1,1]]
THRESH_GRID = np.arange(0.20, 0.81, 0.02)

def _select_threshold(model, Xtr, ytr, blocks, inner):
    oof = np.zeros(len(ytr))
    for itr, ite in inner.split(Xtr, ytr):
        p = make_pipe(clone(model), blocks).fit(Xtr.iloc[itr], ytr[itr])

```

```

        oof[ite] = p.predict_proba(Xtr.iloc[ite])[:,1]
    best_t, best_f = 0.5, -1
    for t in THRESH_GRID:
        f = f1_score(ytr, (oof>=t).astype(int), average="macro")
        if f > best_f: best_f, best_t = f, t
    return best_t

def _select_ensemble(Xtr, ytr, blocks, inner):
    best=(-1,None,None)
    for sub in CANDIDATE_SUBSETS:
        tags=[t for t in sub if t in ENSEMBLE_POOL]
        if len(tags)<2: continue
        for w in CANDIDATE_WEIGHTS:
            if w is not None and len(w)!=len(tags): continue
            sc=[]
            for itr,ite in inner.split(Xtr,ytr):
                vc=VotingClassifier([(t,ENSEMBLE_POOL[t]()) for t in tags],
                                    voting="soft", weights=w)
                p=make_pipe(vc,blocks).fit(Xtr.iloc[itr],ytr[itr])
                sc.append(f1_score(ytr[ite],p.predict(Xtr.iloc[ite]),average="macro"))
            if np.mean(sc)>best[0]: best=(np.mean(sc),tags,w)
    return best[1], best[2]

def nested_eval(data, labels, blocks, model_name, model, grid, splits,
                 tune_threshold=TUNE_THRESHOLD_DEFAULT):
    """Repeated nested CV. Returns per-outer-fold metrics + OOF arrays."""
    inner = StratifiedKFold(N_INNER_SPLITS, shuffle=True, random_state=RS)
    rows=[]; oof_p=np.full(len(labels),np.nan); oof_pred=np.full(len(labels),-1)
    for tr,te in splits:
        Xtr,Xte = data.iloc[tr], data.iloc[te]; ytr,yte = labels[tr], labels[te]
        if model_name=="Soft Voting":
            tags,w = _select_ensemble(Xtr,ytr,blocks,inner)
            est = VotingClassifier([(t,ENSEMBLE_POOL[t]()) for t in tags],voting="soft", weights=w)
            fitted = make_pipe(est,blocks).fit(Xtr,ytr); base=est
        elif grid:
            gs = GridSearchCV(make_pipe(clone(model),blocks), grid,
                              scoring="f1_macro", cv=inner, n_jobs=-1)
            gs.fit(Xtr,ytr); fitted=gs.best_estimator_; base=fitted.named_steps['estimator']
        else:
            fitted = make_pipe(clone(model),blocks).fit(Xtr,ytr); base=clone(model)
            thr = _select_threshold(base,Xtr,ytr,blocks,inner) if tune_threshold else 0.5
            pp = fitted.predict_proba(Xte)[:,1]; yp=(pp>=thr).astype(int)
            oof_p[te]=pp; oof_pred[te]=yp
            tn,fp,fn,tp = confusion_matrix(yte,yp,labels=[0,1]).ravel()
            rows.append({"Macro-F1":f1_score(yte,yp,average="macro"),
                        "AUC":roc_auc_score(yte,pp) if len(np.unique(yte))>1 else 0.5,
                        "MCC":matthews_corrcoef(yte,yp),
                        "BalAcc":balanced_accuracy_score(yte,yp),
                        "Sens":tp/(tp+fn) if (tp+fn) else np.nan,
                        "Spec":tn/(tn+fp) if (tn+fp) else np.nan,
                        "Brier":brier_score_loss(yte,pp),"thr":thr})
    return pd.DataFrame(rows), oof_p, oof_pred

outer = RepeatedStratifiedKFold(n_splits=N_OUTER_SPLITS, n_repeats=N_OUTER_REPEATS,
                                random_state=RS)

SPLITS = list(outer.split(df,y))
n_te = len(y)/N_OUTER_SPLITS; n_tr = len(y)-n_te
LR = lambda: LogisticRegression(max_iter=5000, class_weight="balanced", random_state=RS)
LRG = {"clf_C":[0.05,0.5,5]}
BLK_ALL = ("clinical","thermal","biomarkers")

```

```

# %% ===== PART 1 BENCHMARK INTEGRITY =====
print("="*78); print("PART 1 BENCHMARK INTEGRITY"); print("="*78)
depth3 = DecisionTreeClassifier(max_depth=3, random_state=RS).fit(df[Rc+Lc].va
print(f" Depth-3 tree on raw temperatures: {depth3.score(df[Rc+Lc].values,y):
print(" Interpretation: well below 1.0, so class labels are NOT a determinist
print(" function of the input features. Recommended as a routine check before
print(" any analysis of a public benchmark.")

# %% ===== PART 2 COHORT STRUCTURE =====
print("\n"+"="*78); print("PART 2 COHORT STRUCTURE (the confound)"); print('
ageCG, ageDM = df.loc[df.group=="CG", "Age"], df.loc[df.group=="DM", "Age"]
print(f" n = {len(df)} CG = {len(ageCG)} DM = {len(ageDM)}")
print(f" Age CG {ageCG.mean():.1f} +/- {ageCG.std():.1f}"
      f" DM {ageDM.mean():.1f} +/- {ageDM.std():.1f}"
      f" difference {ageDM.mean()-ageCG.mean():.1f} years")
u_age, p_age = mannwhitneyu(ageCG, ageDM)
print(f" Age difference: Mann-Whitney p = {p_age:.3g},"
      f" Cliff's delta = {cliffs_delta(ageDM.values, ageCG.values):+.3f}")
ps = LogisticRegression(max_iter=5000).fit(Z[["Age", "BMI"]], yv).predict_proba
ps = np.clip(ps, .02, .98); lg=np.log(ps/(1-ps)); cal=.2*np.std(lg)
i0,i1 = np.where(yv==0)[0], np.where(yv==1)[0]
dd,_ = NearestNeighbors(n_neighbors=1).fit(lg[i1].reshape(-1,1)).kneighbors(lg
n_match = int((dd.ravel()<=cal).sum())
print(f" Propensity (age+BMI): mean CG {ps[yv==0].mean():.3f} DM {ps[yv==1].
print(f" 1:1 caliper matching retains {n_match}/{len(i0)} controls")
print(" -> Poor common support. Covariate adjustment alone extrapolates;")
print(" we therefore also report an age-restricted re-analysis (Part 4).")

# %% ===== PART 3 PRIMARY: INCREMENTAL VALUE OF THERMAL INFORMATION =====
print("\n"+"="*78)
print("PART 3 PRIMARY ANALYSIS -- incremental value over clinical variables
print("="*78)
print(f" Nested CV: {N_OUTER_SPLITS}x{N_OUTER_REPEATS} = {len(SPLITS)} untouc
      f" {N_INNER_SPLITS}-fold inner; threshold fixed at 0.5.\n")
BLOCKS = OrderedDict([("Clinical only", ("clinical",)),
                      ("Thermal only", ("thermal",)),
                      ("Clinical + Thermal", ("clinical", "thermal")),
                      ("Clinical + Thermal + Biomarkers", BLK_ALL)])
blk_scores, blk_rows = {}, []
for lbl, blk in BLOCKS.items():
    t0=time.time()
    mt,_ = nested_eval(df, y, blk, "Logistic Regression", LR(), LRG, SPLITS)
    blk_scores[lbl]=mt["Macro-F1"].values; m,c = ci95(mt["Macro-F1"].values)
    blk_rows.append({"Feature block":lbl, "Macro-F1":m, "SD":mt["Macro-F1"].std(
        "CI_lo":c[0], "CI_hi":c[1], "AUC":mt["AUC"].mean()})
    print(f" {lbl:<34s} F1 = {m:.4f} [{c[0]:.4f}, {c[1]:.4f}]"
          f" AUC = {mt['AUC'].mean():.4f} [{time.time()-t0:.0f}s]")
blk_tab = pd.DataFrame(blk_rows); blk_tab.to_csv("A3_IncrementalValue.csv", ir

print("\n Paired incremental tests (Nadeau-Bengio corrected):")
INCR = []
for a_l,b_l in [("Clinical + Thermal", "Clinical only"),
                ("Clinical + Thermal + Biomarkers", "Clinical + Thermal"),
                ("Clinical + Thermal + Biomarkers", "Clinical only")]:
    d = blk_scores[a_l]-blk_scores[b_l]; tt,pp = nadeau_bengio(d, n_tr, n_te)
    boot = [np.mean(np.random.RandomState(i).choice(d, len(d), True)) for i in r
    lo,hi = np.percentile(boot, [2.5, 97.5])
    INCR.append({"comparison":f"{a_l} vs {b_l}", "delta_pp":d.mean()*100,
                "CI_lo_pp":lo*100, "CI_hi_pp":hi*100, "NB_p":pp})
    print(f" {a_l} vs {b_l}:")
    print(f" {d.mean()*100:+.2f} pp 95% CI [{lo*100:+.2f}, {hi*100:+.2f}]

```

```

f"    -> {'significant' if (pp==pp and pp<.05) else 'NOT significant'}
pd.DataFrame(INCR).to_csv("A3_IncrementalTests.csv", index=False)

# %% ===== PART 4 PRIMARY: HOW MUCH OF THE ACCURACY IS AGE? =====
print("\n"+"="*78)
print("PART 4 PRIMARY ANALYSIS -- age-restricted re-analysis")
print("="*78)
mb = ((df["Age"]>=AGE_LO)&(df["Age"]<=AGE_HI)).values
sub = df.loc[mb].reset_index(drop=True); ysub = y[mb]
print(f" Common-support cohort ({AGE_LO}-{AGE_HI} y): n = {len(sub)}"
      f" CG = {int((ysub==0).sum())} DM = {int((ysub==1).sum())}")
print(f" residual age gap = {sub.loc[ysub==1,'Age'].mean()-sub.loc[ysub==0,'Age'].mean()} (the cohort cannot be fully age-balanced)")
sub_splits = list(RepeatedStratifiedKFold(n_splits=N_OUTER_SPLITS,
                                           n_repeats=N_OUTER_REPEATS, random_state=RS).split(sub, ysub))
mt_full,_,_ = nested_eval(df, y, BLK_ALL, "Logistic Regression", LR(), LRG, SF)
mt_band,_,_ = nested_eval(sub, ysub, BLK_ALL, "Logistic Regression", LR(), LRG, SF)
f_full, c_full = ci95(mt_full["Macro-F1"].values)
f_band, c_band = ci95(mt_band["Macro-F1"].values)
DROP = (f_band - f_full)*100
print(f"\n Full cohort macro-F1 = {f_full:.4f} [{c_full[0]:.4f}, {c_full[1]:.4f}]"
      f" Age-restricted cohort macro-F1 = {f_band:.4f} [{c_band[0]:.4f}, {c_band[1]:.4f}]"
      f" DROP = {DROP:+.2f} percentage points")
print("\n This is the audit's headline quantity: the share of apparent")
print(" performance that disappears once the non-overlapping age range is removed")
pd.DataFrame([{"cohort": "full", "F1": f_full, "lo": c_full[0], "hi": c_full[1]},
              {"cohort": f"age {AGE_LO}-{AGE_HI}", "F1": f_band, "lo": c_band[0], "hi": c_band[1]},
              {"cohort": "drop_pp", "F1": DROP, "lo": np.nan, "hi": np.nan}]).to_csv("A4_AgeRestricted.csv", index=False)

# %% ===== PART 5 PRIMARY: ERROR MECHANISM (formal test) =====
print("\n"+"="*78)
print("PART 5 PRIMARY ANALYSIS -- are errors concentrated on age-atypical controls")
print("="*78)
mt_err, oofp_err, oofpred_err = nested_eval(df, y, BLK_ALL, "Logistic Regression", LR(), LRG, SPLITS)
mis = np.where(oofpred_err != y)[0]; cor = np.where(oofpred_err == y)[0]
age = df["Age"].values
print(f" Misclassified: {len(mis)}/{len(y)} ({100*len(mis)/len(y):.1f}%)")
print(f" Mean age correct {age[cor].mean():.1f} misclassified {age[mis].mean():.1f}"
      f" difference {age[mis].mean()-age[cor].mean():+.1f} y")
if len(mis) >= 3:
    u_e, p_e = mannwhitneyu(age[mis], age[cor])
    d_e = cliffs_delta(age[mis], age[cor])
    rb = np.random.RandomState(RS)
    bs = [cliffs_delta(rb.choice(age[mis], len(mis), True), rb.choice(age[cor], len(cor), True))
           for _ in range(2000)]
    lo_e, hi_e = np.percentile(bs, [2.5, 97.5])
    print(f" FORMAL TEST Mann-Whitney p = {p_e:.4f}")
    print(f" Cliff's delta = {d_e:+.3f} [{lo_e:+.3f}, {hi_e:+.3f}]")
    print(f" -> {'Errors ARE significantly age-atypical' if p_e < .05 else 'NOT'}")
# direction breakdown
cg_as_dm = mis[(y[mis]==0)]; dm_as_cg = mis[(y[mis]==1)]
print(f"\n Controls predicted diabetic : n={len(cg_as_dm)}"
      f" mean age {age[cg_as_dm].mean():.1f}"
      f" (vs {age[y==0].mean():.1f} for all controls)" if len(cg_as_dm) > 0 else ""
print(f" Diabetics predicted control : n={len(dm_as_cg)}"
      f" mean age {age[dm_as_cg].mean():.1f}"
      f" (vs {age[y==1].mean():.1f} for all diabetics)" if len(dm_as_cg) > 0 else ""
print("\n Mechanism: the classifier fails precisely where age stops being")
print(" informative - older controls and younger diabetics.")

```



```

pd.DataFrame({"Age":age[mis], "TAI":A["TAI"].values[mis],
              "true": [classes[v] for v in y[mis]],
              "pred": [classes[v] for v in oofpred_err[mis]],
              "confidence": np.maximum(oofp_err[mis], 1-oofp_err[mis])})
.to_csv("A5_ErrorCases.csv", index=False)
pd.DataFrame([{"mean_age_correct":age[cor].mean(), "mean_age_misclassified"
              "MWU_p":p_e, "cliffs_delta":d_e, "ci_lo":lo_e, "ci_hi":hi_e}])
.to_csv("A5_ErrorAgeTest.csv", index=False)

print("\n"+"="*78); print("PART 6 BIOMARKER VALIDATION"); print("="*78)
rows,praw=[],[]; rb=np.random.RandomState(RS)
for k in BIO:
    cg,dm = B[k][df.group=="CG"], B[k][df.group=="DM"]
    _,p = mannwhitneyu(cg,dm); praw.append((k,p)); d=cliffs_delta(dm,cg)
    bs=[cliffs_delta(rb.choice(dm,len(dm),True), rb.choice(cg,len(cg),True)) for _ in range(1000)]
    rows.append({"Biomarker":k, "CG":cg.mean(), "DM":dm.mean(), "Cliff d":d,
                "CI lo":np.percentile(bs,2.5), "CI hi":np.percentile(bs,97.5),
                "size":dlab(d), "raw p":p})
hp=holm(praw); unadj=pd.DataFrame(rows); unadj["Holm p"]=unadj["Biomarker"].map(hp)
unadj["sig"]=np.where(unadj["Holm p"]<.05, "YES", "no")
print("\n 6a Unadjusted:"); print(unadj.round(5).to_string(index=False))
unadj.to_csv("A6a_Unadjusted.csv", index=False)

def firth_table(Zf, yf, tag):
    rws,pr=[],[]
    for k in BIO:
        Xf=Zf[["Age", "BMI", k]].values
        b,se,_=firth_logit(Xf,yf); st,p=firth_plrt(Xf,yf,2); pr.append((k,p))
        rws.append({"Biomarker":k, "adj OR/SD":np.exp(b[3]),
                    "CI lo":np.exp(b[3]-1.96*se[3]), "CI hi":np.exp(b[3]+1.96*se[3]),
                    "raw p":p})
    h=holm(pr); t=pd.DataFrame(rws); t["Holm p"]=t["Biomarker"].map(h)
    t["sig@0.05"]=np.where(t["Holm p"]<.05, "YES", "no")
    print(f"\n {tag}"); print(t.round(4).to_string(index=False))
    return t, int((t["sig@0.05"]=="YES").sum())

t_full, n_full = firth_table(Z, yv, "6b Full cohort, Firth-adjusted for Age+BMI")
t_full.to_csv("A6b_Firth_Full.csv", index=False)
Ab = A.loc[mb].reset_index(drop=True); Zb = Ab.copy()
for c in BIO+["Age", "BMI"]:
    sd=Ab[c].std(ddof=0); Zb[c]=(Ab[c]-Ab[c].mean())/(sd if sd>0 else 1.0)
t_band, n_band = firth_table(Zb, Zb["DM"].values,
                             f"6c Common support {AGE_LO}-{AGE_HI} (n={len(Ab)})")
t_band.to_csv("A6c_Firth_Restricted.csv", index=False)
llc = firth_logit(Z[["Age", "BMI"]].values, yv)[2]
lla = firth_logit(Z[["Age", "BMI"]]+BIO].values, yv)[2]
lr_s = max(2*(lla-llc),0); lr_p = chi2.sf(lr_s,5)
print(f"\n 6d Nested LRT (panel beyond Age+BMI): chi2(5) = {lr_s:.3f} p = {lr_p:.3f}")
print(f" -> {'panel adds explanatory information' if lr_p<.05 else 'no explanatory information'}")
print(f"\n Significant after adjustment: full {n_full}/5, common-support {n_band}/5")

mdl_f1, mdl_rows, mdl_oofp, mdl_oofpred = {}, [], {}, {}
if RUN_FULL_MODEL_COMPARISON:
    print("\n"+"="*78); print("PART 7 SECONDARY -- model comparison"); print("="*78)
    MODELS = OrderedDict()
    MODELS["Logistic Regression"] = (LR(), LRG)
    MODELS["SVM"] = (SVC(probability=True, class_weight="balanced", random_state=RS),
                     {"clf_C": [0.5, 5]})
    MODELS["Random Forest"] = (RandomForestClassifier(random_state=RS, class_weight="balanced",

```

```

        {"clf__n_estimators": [200], "clf__max_depth": [4,
if XGB:
    MODELS["XGBoost"] = (XGBClassifier(random_state=RS, eval_metric="logloss",
        {"clf__n_estimators": [300], "clf__max_depth": [2, 4]
for nm, (md, gr) in MODELS.items():
    t0=time.time(); mt, op, opd = nested_eval(df, y, BLK_ALL, nm, md, gr, SPLITS)
    mdl_f1[nm]=mt["Macro-F1"].values; mdl_oofp[nm]=op; mdl_oofpred[nm]=opd
    m, c=ci95(mt["Macro-F1"].values)
    mdl_rows.append({"Model": nm, "Macro-F1": m, "CI_lo": c[0], "CI_hi": c[1],
        "AUC": mt["AUC"].mean(), "MCC": mt["MCC"].mean(),
        "BalAcc": mt["BalAcc"].mean(), "Sens": mt["Sens"].mean(),
        "Spec": mt["Spec"].mean(), "Brier": mt["Brier"].mean()})
    print(f" {nm:<20s} F1={m:.4f} [{c[0]:.4f}, {c[1]:.4f}] AUC={mt['AUC'].mean():.4f}
        f" [{time.time()-t0:.0f}s]")
    t0=time.time(); mt, op, opd = nested_eval(df, y, BLK_ALL, "Soft Voting", None, None)
    mdl_f1["Soft Voting"]=mt["Macro-F1"].values; mdl_oofp["Soft Voting"]=op
    mdl_oofpred["Soft Voting"]=opd; m, c=ci95(mt["Macro-F1"].values)
    mdl_rows.append({"Model": "Soft Voting", "Macro-F1": m, "CI_lo": c[0], "CI_hi": c[1],
        "AUC": mt["AUC"].mean(), "MCC": mt["MCC"].mean(),
        "BalAcc": mt["BalAcc"].mean(), "Sens": mt["Sens"].mean(),
        "Spec": mt["Spec"].mean(), "Brier": mt["Brier"].mean()})
    print(f" {'Soft Voting':<20s} F1={m:.4f} [{c[0]:.4f}, {c[1]:.4f}]
        f" AUC={mt['AUC'].mean():.4f} [{time.time()-t0:.0f}s]")

R_ = pd.DataFrame(mdl_rows).sort_values("Macro-F1", ascending=False).reset_index()
print("\n"+R_.round(4).to_string(index=False)); R_.to_csv("A7_Models.csv", index=False)
best = R_.iloc[0]["Model"]
fs_, fp_ = friedmanchisquare(*[mdl_f1[n] for n in R_["Model"]])
ref = R_.iloc[1]["Model"]
tt, pp_ = nadeau_bengio(mdl_f1[best]-mdl_f1[ref], n_tr, n_te)
print(f"\n Friedman chi2={fs_:.3f} p={fp_:.4f}"
    f" -> {'some algorithms differ' if fp_<.05 else 'no overall difference'}")
print(f" {best} vs {ref}: {(mdl_f1[best].mean()-mdl_f1[ref].mean())*100:.4f}%"
    f" NB p={pp_:.4f} Cliff d={cliffs_delta(mdl_f1[best], mdl_f1[ref]):.4f}")
print(" NOTE: an omnibus difference does not imply the top models differ;")
print(" report the pairwise result alongside it.")

print("\n Threshold sensitivity (Logistic Regression):")
mt_t, _, _ = nested_eval(df, y, BLK_ALL, "Logistic Regression", LR(), LRG, SPLITS,
    tune_threshold=True)
print(f" fixed 0.5 F1 = {mdl_f1['Logistic Regression'].mean():.4f}")
print(f" inner-CV selected F1 = {mt_t['Macro-F1'].mean():.4f}")
print(" -> reported for transparency; tuning did not help at this sample size")
else:
    print("\n [PART 7 skipped: RUN_FULL_MODEL_COMPARISON = False]")
    best = "Logistic Regression"
    mdl_f1[best] = blk_scores["Clinical + Thermal + Biomarkers"]
    _, mdl_oofp[best], mdl_oofpred[best] = nested_eval(df, y, BLK_ALL, best, LR(), LRG, SPLITS,
        tune_threshold=True)

print("\n"+"="*78); print("PART 8 ATTRIBUTION (SHAP, interpretation only)");
if SHAP:
    for tag, blk in [("combined", BLK_ALL), ("thermal-only", ("thermal", "biomarkers"))]:
        fit = make_pipe(RandomForestClassifier(n_estimators=400, class_weight="balanced",
            random_state=RS), blk).fit(df, y)
        nmz = fit.named_steps["feat"].feature_names_
        Xb = pd.DataFrame(fit.named_steps["scale"].transform(
            fit.named_steps["feat"].transform(df)), columns=nmz)
        sv = shap.TreeExplainer(fit.named_steps["clf"]).shap_values(Xb)
        arr = sv[1] if isinstance(sv, list) else (sv[:, :, 1] if getattr(sv, "ndim", 0) > 2 else sv)
        imp=(pd.DataFrame({"Feature": nmz, "mean|SHAP|": np.abs(arr).mean(0)}))

```

```

        .sort_values("mean|SHAP|",ascending=False).reset_index(drop=True)
    print(f"\n [{tag}] top 8:"); print(imp.head(8).round(4).to_string(indent=2))
    imp.to_csv(f"A8_SHAP_{tag}.csv",index=False)
    shap.summary_plot(arr,Xb,plot_type="bar",show=False)
    plt.title(f"SHAP - {tag}",fontweight="bold"); plt.tight_layout()
    plt.savefig(f"A8_SHAP_{tag}.png",dpi=300,bbox_inches="tight"); plt.show()
    print("\n The thermal-only model is reported because Age dominates the")
    print(" combined attribution; it shows which plantar regions contribute")
    print(" when clinical variables are removed.")
else:
    print(" shap not installed")

plt.figure(figsize=(8,4.2))
sns.histplot(data=df, x="Age", hue="group", bins=25,
             palette={"CG":"#3498db","DM":"#e74c3c"}, alpha=.65, element="step")
plt.axvspan(AGE_LO,AGE_HI,color="grey",alpha=.15)
plt.title("Age distribution by group (shaded = common support)",fontweight="bold")
plt.tight_layout(); plt.savefig("A_Fig1_Age.png",dpi=300,bbox_inches="tight");

plt.figure(figsize=(9,4.4))
e=[blk_tab["Macro-F1"]-blk_tab["CI_lo"], blk_tab["CI_hi"]-blk_tab["Macro-F1"]]
plt.barh(blk_tab["Feature block"],blk_tab["Macro-F1"],xerr=e,capsize=4,
        color="#3498db",edgecolor="k",alpha=.88)
plt.gca().invert_yaxis(); plt.xlim(0,1)
plt.xlabel(f"Macro-F1 (nested CV, {len(SPLITS)} outer folds, 95% CI)")
plt.title("Incremental value of thermal information",fontweight="bold")
plt.tight_layout(); plt.savefig("A_Fig2_Incremental.png",dpi=300,bbox_inches="tight");

plt.figure(figsize=(6.5,4.4))
plt.bar(["Full cohort",f"Age {AGE_LO}-{AGE_HI}"],[f_full,f_band],
       yerr=[[f_full-c_full[0],f_band-c_band[0]],[c_full[1]-f_full,c_band[1]-f_band]],
       capsize=6,color=["#3498db","#e67e22"],edgecolor="k",alpha=.9)
plt.ylabel("Macro-F1 (nested CV, 95% CI)"); plt.ylim(0,1.05)
plt.title(f"Performance loss on removing non-overlapping ages ({DROP:+.1f} pp)",fontweight="bold")
plt.tight_layout(); plt.savefig("A_Fig3_AgeRestricted.png",dpi=300,bbox_inches="tight");

if len(mis):
    plt.figure(figsize=(7,4.4))
    sns.boxplot(data=pd.DataFrame({"Age":np.concatenate([age[cor],age[mis]]),
                                   "Prediction":["correct"]*len(cor)+["misclassified"]*len(mis)}),
               x="Prediction",y="Age",palette={"correct":"#27ae60","misclassified":"#e74c3c"},
               sns.stripplot(data=pd.DataFrame({"Age":np.concatenate([age[cor],age[mis]]),
                                                  "Prediction":["correct"]*len(cor)+["misclassified"]*len(mis)}),
                             x="Prediction",y="Age",color="black",size=3,alpha=.4)
    plt.title("Errors concentrate on age-atypical subjects",fontweight="bold")
    plt.tight_layout(); plt.savefig("A_Fig4_ErrorAge.png",dpi=300,bbox_inches="tight");

for tbl,fn,tt_ in [(t_full,"A_Fig5_Forest_Full.png","Full cohort"),
                  (t_band,"A_Fig6_Forest_Restricted.png",f"Age {AGE_LO}-{AGE_HI}"),
                  (ok,"A_Fig7_Forest_Adjusted.png",f"Age {AGE_LO}-{AGE_HI} + BMI")]:
    ok=tbl.dropna(subset=["CI_lo"])
    plt.figure(figsize=(7.5,4.2)); yy=np.arange(len(ok))
    plt.errorbar(ok["adj OR/SD"],yy,
                xerr=[np.maximum(ok["adj OR/SD"]-ok["CI_lo"],0),
                     np.maximum(ok["CI_hi"]-ok["adj OR/SD"],0)],
                fmt="o",color="#2c7fb8",capsize=4,lw=1.4)
    plt.axvline(1,ls="--",color="k"); plt.yticks(yy,ok["Biomarker"]); plt.xscale("log")
    plt.xlabel("Firth adjusted OR per SD (log scale), 95% CI")
    plt.title(f"Biomarker ORs adjusted for Age+BMI - {tt_}",fontweight="bold")
    plt.tight_layout(); plt.savefig(fn,dpi=300,bbox_inches="tight"); plt.show()

```

```

print("\n"+"="*78); print("PART 9  AUDIT SUMMARY  (headline numbers for the pa
print(f"  Benchmark integrity      : depth-3 tree {depth3.score(df[Rc+Lc].valu
print(f"  Age imbalance            : {ageDM.mean()-ageCG.mean():.1f} y, MWU p
print(f"  Propensity overlap        : {n_match}/{len(i0)} controls matchable")
print(f"  Clinical only              : macro-F1 {blk_tab.iloc[0]['Macro-F1']:.4f
print(f"  Thermal only                : macro-F1 {blk_tab.iloc[1]['Macro-F1']:.4f
print(f"  + thermal over clinical    : {INCR[0]['delta_pp']+.2f} pp, p = {INCR[
print(f"  + biomarkers               : {INCR[1]['delta_pp']+.2f} pp, p = {INCR[
print(f"  Age-restricted drop        : {DROP+.2f} pp")
if len(mis) >= 3:
    print(f"  Error age difference      : {age[mis].mean()-age[cor].mean():+.1f
print(f"  Biomarkers adjusted          : full {n_full}/5, common-support {n_band}/
print(f"  Panel LRT beyond Age+BMI : p = {lr_p:.4f}")
print("\n CONCLUSION: on this benchmark, plantar thermal features provide no"
print("  measurable diagnostic information beyond age and BMI, and no thermal"
print("  biomarker retains an independent association after age adjustment.")
print("  Age-balanced cohorts and external validation are required.")
print("="*78)

from sklearn.model_selection import StratifiedKFold as _SKF, cross_val_predict
from sklearn.neighbors import NearestNeighbors as _NN
from sklearn.calibration import calibration_curve
try:
    from statsmodels.stats.contingency_tables import mcnemar as sm_mcnemar
except Exception:
    sm_mcnemar = None

def _pipe_block(blocks):
    return Pipeline([("feat",Features(blocks)),("scale",StandardScaler()),
                     ("clf",LogisticRegression(max_iter=5000,class_weight="bal
_cv5 = _SKF(5, shuffle=True, random_state=RS)
BLOCKS = OrderedDict([("Clinical",("clinical",)),
                      ("Clinical+Thermal",("clinical","thermal")),
                      ("Clinical+Thermal+Biomarkers",("clinical","thermal","bi
OOF = {l:_cvp(_pipe_block(b), df, y, cv=_cv5, method="predict_proba")[:,1] fo
PRED = {l:(OOF[l]>=.5).astype(int) for l in BLOCKS}
age = df["Age"].values
bmi = SimpleImputer(strategy="median").fit_transform(df[["IMC"]]).ravel()
_PSX = np.column_stack([(age-age.mean())/age.std(), (bmi-bmi.mean())/bmi.std()
ps = np.clip(LogisticRegression(max_iter=5000).fit(_PSX, y).predict_proba(_PSX
lg = np.log(ps/(1-ps)); caliper = 0.2*np.std(lg)
i_cg, i_dm = np.where(y==0)[0], np.where(y==1)[0]
def matched_indices(seed):
    rng=np.random.RandomState(seed); order=rng.permutation(i_cg); used=set();
    nn=_NN(n_neighbors=min(5,len(i_dm))).fit(lg[i_dm].reshape(-1,1))
    dist,idx=nn.kneighbors(lg[order].reshape(-1,1))
    for r,c in enumerate(order):
        for d,j in zip(dist[r],idx[r]):
            cand=i_dm[j]
            if d<=caliper and cand not in used: used.add(cand); pairs.append(
    return np.array([p for pr in pairs for p in pr]) if pairs else None
matched_idx = matched_indices
def wauc(yt, sc, w):
    pos=np.where(yt==1)[0]; neg=np.where(yt==0)[0]; num=0.0; den=0.0
    for i in pos:
        for j in neg:
            wij=w[i]*w[j]; num += wij*((sc[i]>sc[j])+0.5*(sc[i]==sc[j])); den+

```

```

    return num/den if den>0 else np.nan
def wf1(yt, yp, w):
    def wprf(cls):
        tp=np.sum(w[(yp==cls)&(yt==cls)]); fp=np.sum(w[(yp==cls)&(yt!=cls)]);
        p=tp/(tp+fp) if (tp+fp)>0 else 0; r=tp/(tp+fn) if (tp+fn)>0 else 0
        return 0 if (p+r)==0 else 2*p*r/(p+r)
    return 0.5*(wprf(0)+wprf(1))

print("="*76); print("A PROPENSITY MODEL + COMMON SUPPORT"); print("="*76)
i_cg,i_dm = np.where(y==0)[0], np.where(y==1)[0]
lo_ov = max(ps[i_cg].min(), ps[i_dm].min())
hi_ov = min(ps[i_cg].max(), ps[i_dm].max())
in_support = (ps>=lo_ov)&(ps<=hi_ov)
print(f" PS range CG [{ps[i_cg].min():.3f}, {ps[i_cg].max():.3f}]"
      f" DM [{ps[i_dm].min():.3f}, {ps[i_dm].max():.3f}]")
print(f" Common-support region: PS in [{lo_ov:.3f}, {hi_ov:.3f}]")
print(f" Subjects in common support: {int(in_support.sum())}/{len(y)}"
      f" (CG {int(in_support[i_cg].sum())}/{len(i_cg)}, DM {int(in_support[i_dm].sum())}/{len(i_dm)})")
print(f" Caliper for matching: {caliper:.3f} on logit(PS)")

print("\n"+"="*76); print("B COVARIATE BALANCE (standardized mean differences)"); print("="*76)
def smd(a, b):
    va, vb = np.var(a, ddof=1), np.var(b, ddof=1)
    sp = np.sqrt((va+vb)/2)
    return 0.0 if sp==0 else (np.mean(a)-np.mean(b))/sp

# average matched set over resamples for a stable balance estimate
REPB=200
mm_cg, mm_dm = [], []
for s in range(REPB):
    ii=matched_indices(s)
    if ii is None: continue
    lab=y[ii]; mm_cg.append(ii[lab==0]); mm_dm.append(ii[lab==1])
def pooled(idx_lists): return np.concatenate(idx_lists) if idx_lists else np.array([])
mcg, mdm = pooled(mm_cg), pooled(mm_dm)

rows=[]
for name, vec in [("Age",age),("BMI",bmi),("PropensityScore",ps)]:
    before = smd(vec[i_dm], vec[i_cg])
    after = smd(vec[mdm], vec[mcg]) if len(mdm) and len(mcg) else np.nan
    rows.append({"Covariate":name,"SMD before":before,"SMD after":after,
                "|SMD|<0.1 after":"yes" if (after==after and abs(after)<0.1) else "no"})
bal=pd.DataFrame(rows); print(bal.round(4).to_string(index=False))
bal.to_csv("Q1_BalanceSMD.csv",index=False)
print("\n Convention: |SMD| < 0.1 indicates negligible imbalance after matching")
print(" Age SMD before matching is large (the confounder); after matching it"
      " should fall toward 0, confirming the design removed the age difference")

print("\n"+"="*76)
print("C MATCHED-COHORT DISCRIMINATION (sensitivity analysis; small n)"); print("="*76)
REP=500
rows=[]
for lbl in BLOCKS:
    f1s,aucs=[],[]
    for s in range(REP):
        ii=matched_indices(s)

```



```

        if ii is None or len(np.unique(y[ii]))<2: continue
        f1s.append(f1_score(y[ii],(OOF[lbl][ii]>=.5).astype(int),average="macro"))
        try: aucs.append(roc_auc_score(y[ii],OOF[lbl][ii]))
        except Exception: pass
    f1s,aucls=np.array(f1s),np.array(aucs)
    rows.append({"Block":lbl,"F1 median":np.median(f1s),
                "F1 CI":f"[{np.percentile(f1s,2.5):.3f}, {np.percentile(f1s,97.5):.3f}]",
                "AUC median":np.median(aucs),
                "AUC CI":f"[{np.percentile(aucs,2.5):.3f}, {np.percentile(aucs,97.5):.3f}]"}
    mtab=pd.DataFrame(rows); print(mtab.to_string(index=False)); mtab.to_csv("Q1_MatchedCalibration.csv",index=False)
    print(f"\n Based on ~{len(mdm)//REPB if REPB else 0}-pair matched sets from {len(mdm)}
    print(" matchable controls. Interpret as SUGGESTIVE, not confirmatory: the")
    print(" matchable subset is small and reused across resamples.")

print("\n"+"="*76); print("D MATCHED-COHORT CALIBRATION + DECISION CURVE");

rep_ii = np.unique(np.concatenate([matched_indices(s) for s in range(REPB)
                                   if matched_indices(s) is not None]))
ym = y[rep_ii]
def ece(yt,pp,nb=10):
    conf=np.maximum(pp,1-pp); pc=(pp>=.5).astype(int); corr=(pc==yt).astype(float)
    bins=np.linspace(0,1,nb+1); e=0.0
    for lo,hi in zip(bins[:-1],bins[1:]):
        m=(conf>lo)&(conf<=hi)
        if m.sum(): e+=(m.sum()/len(yt))*abs(corr[m].mean()-conf[m].mean())
    return e
crows=[]
for lbl in BLOCKS:
    pm=OOF[lbl][rep_ii]
    crows.append({"Block":lbl,"n":len(rep_ii),"Brier":brier_score_loss(ym,pm),
                "AUC":roc_auc_score(ym,pm) if len(np.unique(ym))>1 else np.nan})
cm=pd.DataFrame(crows); print("\n Calibration on common-support matched cohort")
print(cm.round(4).to_string(index=False)); cm.to_csv("Q1_MatchedCalibration.csv",index=False)

def net_benefit(yt,pp,pt):
    N=len(yt); tp=np.sum((pp>=pt)&(yt==1)); fp=np.sum((pp>=pt)&(yt==0))
    return 0.0 if pt>=1 else tp/N - fp/N*(pt/(1-pt))
pts=np.linspace(0.05,0.75,29)
dca={l:[net_benefit(ym,OOF[l][rep_ii],t) for t in pts] for l in BLOCKS}
nb_all=[net_benefit(ym,np.ones_like(ym,dtype=float),t) for t in pts]
print("\n Matched-cohort net benefit:")
print(" pt "+" ".join(f"{l[:13]:>15s}" for l in list(BLOCKS)+["TreatAll"]))
for t in [0.1,0.3,0.5]:
    k=int(np.argmax(np.abs(pts-t)))
    print(f" {t:.2f}"+" ".join(f"{dca[l][k]:>15.4f}" for l in BLOCKS)+f"{nb_all[k]:>15.4f}")
pd.DataFrame({"threshold":pts,**{l:dca[l] for l in BLOCKS},"TreatAll":nb_all})
    ).to_csv("Q1_MatchedDCA.csv",index=False)

fig,ax=plt.subplots(1,2,figsize=(12,4.4))

covs=bal["Covariate"]; yb=np.arange(len(covs))
ax[0].scatter(bal["SMD before"].abs(),yb,color="#e74c3c",label="Before",s=60,zorder=1)
ax[0].scatter(bal["SMD after"].abs(),yb,color="#27ae60",label="After",s=60,zorder=2)
for i in yb: ax[0].plot([abs(bal['SMD before'][i]),abs(bal['SMD after'][i])],
                        [abs(bal['SMD before'][i]),abs(bal['SMD after'][i])],
                        color="k",lw=1,label="|SMD|=0.1")
ax[0].axvline(0.1,ls="--",color="k",lw=1,label="|SMD|=0.1")
ax[0].set_yticks(yb); ax[0].set_yticklabels(covs); ax[0].set_xlabel("|Standardized Difference|")
ax[0].set_title("Covariate balance (love plot)",fontweight="bold"); ax[0].legend()

ax[1].hist(ps[i_cg],bins=20,alpha=.55,color="#3498db",label="CG",density=True)

```

```

ax[1].hist(ps[i_dm],bins=20,alpha=.55,color="#e74c3c",label="DM",density=True)
ax[1].axvspan(lo_ov,hi_ov,color="grey",alpha=.15,label="Common support")
ax[1].set_xlabel("Propensity score"); ax[1].set_ylabel("Density")
ax[1].set_title("Propensity overlap",fontweight="bold"); ax[1].legend(fontsize=12)
plt.tight_layout(); plt.savefig("Q1_Balance_Overlap.png",dpi=300,bbox_inches='tight')

print("\n"+"="*76); print("E MATCHED-PATH SUMMARY"); print("="*76)
print(" Full cohort (n=167, CG 45 / DM 122)")
print(" -> propensity model on Age + BMI")
print(f" -> common support PS in [{lo_ov:.3f}, {hi_ov:.3f}] ({int(in_supp)} in support)")
print(f" -> 1:1 caliper match ({caliper:.3f}) -> ~{len(set(mcg))} matched pairs")
print(f" -> Age |SMD|: {abs(bal.loc[0,'SMD before']):.2f} before -> "
      f"{abs(bal.loc[0,'SMD after']):.2f} after")
print(" -> discrimination re-evaluated on matched cohort (Section C)")
print("\n KEY: matching balances age (SMD -> ~0). Any discrimination that")
print(" remains in the matched cohort is age-independent. Because the matched")
print(" subset is small, this is reported as a sensitivity analysis motivating")
print(" external, age-balanced validation - not as a primary performance claim")
print("\n Files: Q1_BalanceSMD.csv, Q1_MatchedDiscrim.csv, Q1_MatchedCalibration.csv,")
print(" Q1_MatchedDCA.csv, Q1_Balance_Overlap.png")
print("="*76)

print("="*76); print("1 REPEATED PS MATCHING (median dAUC/dF1 with 95% CI)")
def matched_idx(seed):
    rng=np.random.RandomState(seed); order=rng.permutation(i_cg); used=set();
    nn=NearestNeighbors(n_neighbors=min(5,len(i_dm))).fit(lg[i_dm].reshape(-1,1))
    dist,idx=nn.kneighbors(lg[order].reshape(-1,1))
    for r,c in enumerate(order):
        for d_,j_ in zip(dist[r],idx[r]):
            cand=i_dm[j_]
            if d_<=caliper and cand not in used: used.add(cand); pairs.append((cand,r))
    return np.array([p for pr in pairs for p in pr]) if pairs else None

REP=1000
dA_ct, dF_ct, dA_cb, dF_cb = [],[],[],[]
for s in range(REP):
    ii=matched_idx(s)
    if ii is None or len(np.unique(y[ii]))<2: continue
    yt=y[ii]
    aC=roc_auc_score(yt,OOE["Clinical"][ii]); aT=roc_auc_score(yt,OOE["Clinical+Thermal+Biomarkers"][ii])
    aB=roc_auc_score(yt,OOE["Clinical+Thermal+Biomarkers"][ii])
    fC=f1_score(yt,PRED["Clinical"][ii],average="macro")
    fT=f1_score(yt,PRED["Clinical+Thermal"][ii],average="macro")
    fB=f1_score(yt,PRED["Clinical+Thermal+Biomarkers"][ii],average="macro")
    dA_ct.append(aT-aC); dF_ct.append(fT-fC); dA_cb.append(aB-aC); dF_cb.append(fB-fC)
def summ(a): a=np.array(a); return a.mean(),np.percentile(a,2.5),np.percentile(a,97.5)
for lbl,dA,dF in [("Clinical+Thermal - Clinical",dA_ct,dF_ct),
                  ("Clinical+Thermal+Biomarkers - Clinical",dA_cb,dF_cb)]:
    mA,lA,hA=summ(dA); mF,lF,hF=summ(dF)
    print(f"\n {lbl}:")
    print(f" dAUC median {mA:+.4f} 95% CI [{lA:+.4f}, {hA:+.4f}]")
    print(f" -> {'excludes 0' if (lA>0 or hA<0) else 'includes 0'}")
    print(f" dF1 median {mF*100:+.2f}pp 95% CI [{lF*100:+.2f}, {hF*100:+.2f}]")
    print(f" -> {'excludes 0' if (lF>0 or hF<0) else 'includes 0'}")
pd.DataFrame({"dAUC_CT":dA_ct,"dF1_CT":dF_ct,"dAUC_CB":dA_cb,"dF1_CB":dF_cb}).to_csv("W1_RepeatedMatching.csv",index=False)

```

```

print("\n"+"="*76); print("2 OVERLAP (ATO) WEIGHTING (all 167 subjects)");
# overlap weights: treated w=1-ps, control w=ps -> emphasises the region of
# common support; exact balance on the PS by construction; no extreme weights.
w_ov=np.where(y==1, 1-ps, ps)
print(f" Overlap weights: range [{w_ov.min():.3f}, {w_ov.max():.3f}] (bounded)")
# check balance
def wmean(v,w,mask): return np.sum(v[mask]*w[mask])/np.sum(w[mask])
for name,v in [("Age",age),("BMI",bmi)]:
    b=(wmean(v,w_ov,y==1)-wmean(v,w_ov,y==0))/np.sqrt((v[y==1].var()+v[y==0].var()))
    print(f" weighted SMD {name}: {b:+.3f}")
rows=[]
for l in BLOCKS:
    A=wauc(y,OOF[l],w_ov); F=wf1(y,PRED[l],w_ov)
    rows.append({"Block":l,"overlap-wtd AUC":A,"overlap-wtd F1":F})
wt=pd.DataFrame(rows); print("\n "+wt.round(4).to_string(index=False)); wt.to_csv("wt.csv")
dA_ov=wauc(y,OOF["Clinical+Thermal"],w_ov)-wauc(y,OOF["Clinical"],w_ov)
dA_ovb=wauc(y,OOF["Clinical+Thermal+Biomarkers"],w_ov)-wauc(y,OOF["Clinical"],w_ov)
# bootstrap CI on overlap-weighted dAUC
rng=np.random.RandomState(RS); bA,bB=[],[]
for _ in range(1000):
    ix=rng.choice(len(y),len(y),True)
    if len(np.unique(y[ix]))<2: continue
    bA.append(wauc(y[ix],OOF["Clinical+Thermal"][ix],w_ov[ix])-wauc(y[ix],OOF["Clinical"],w_ov[ix]))
    bB.append(wauc(y[ix],OOF["Clinical+Thermal+Biomarkers"][ix],w_ov[ix])-wauc(y[ix],OOF["Clinical"],w_ov[ix]))
print(f"\n Overlap-weighted dAUC (Thermal-Clinical) = {dA_ov:+.4f}")
print(f" 95% CI [{np.percentile(bA,2.5):+.4f}, {np.percentile(bA,97.5):+.4f}]")
print(f" Overlap-weighted dAUC (+Biomarkers-Clin.) = {dA_ovb:+.4f}")
print(f" 95% CI [{np.percentile(bB,2.5):+.4f}, {np.percentile(bB,97.5):+.4f}]")
print(" Overlap weighting is the most stable estimand here: it uses all")
print(" subjects, avoids extreme weights, and exactly balances the PS.")

print("\n"+"="*76); print("3 IPTW (stabilised + trimmed; SENSITIVITY only)");
p_treat=y.mean()
w_iptw=np.where(y==1, p_treat/ps, (1-p_treat)/(1-ps))
cap=np.percentile(w_iptw,99); w_iptw=np.clip(w_iptw,0,cap)
print(f" stabilised weights range [{w_iptw.min():.2f}, {w_iptw.max():.2f}]")
print(f" (max/median = {w_iptw.max()/np.median(w_iptw):.1f})")
dA_ip=wauc(y,OOF["Clinical+Thermal"],w_iptw)-wauc(y,OOF["Clinical"],w_iptw)
print(f" IPTW-weighted dAUC (Thermal-Clinical) = {dA_ip:+.4f}")
print(" CAUTION: PS near-separation makes IPTW unstable here; reported only")
print(" as a sensitivity check. Overlap weighting (Section 2) is preferred.")
pd.DataFrame([{"design":"overlap","dAUC_CT":dA_ov,"dAUC_CB":dA_ovb},
               {"design":"iptw","dAUC_CT":dA_ip,"dAUC_CB":np.nan}]).to_csv("W3_w.csv")

print("\n"+"="*76); print("4 BOOTSTRAP Clinical+Thermal - Clinical (matched)");
mA,lA,hA=summ(dA_ct); mF,lF,hF=summ(dF_ct)
print(f" Repeated-matching bootstrap (n={len(dA_ct)} matched resamples):")
print(f" dAUC {mA:+.4f} [{lA:+.4f}, {hA:+.4f}] dF1 {mF*100:+.2f}pp [{lF*100:+.2f}, {hF*100:+.2f}]")
print(f" -> {'thermal adds AUC' if lA>0 else 'no confirmed AUC gain (CI includes 0)'}")

print("\n"+"="*76); print("5 PAIRED McNEMAR (Clinical vs Clinical+Thermal)");
def mcnemar_pair(mask=None):
    idx=np.arange(len(y)) if mask is None else mask
    a=PRED["Clinical"][idx]==y[idx]; b=PRED["Clinical+Thermal"][idx]==y[idx]
    n01=int(np.sum(~a&b)); n10=int(np.sum(a&~b))
    tab=[[int(np.sum(a&b)),n10],[n01,int(np.sum(~a&~b))]]
    r=sm.mcnemar(tab,exact=True)
    return n10,n01,r.pvalue

```



```

n10,n01,p=mcnemar_pair()
print(f" Full cohort: clinical-only right/thermal wrong = {n10}, "
      f" thermal right/clinical wrong = {n01}, McNemar p = {p:.4f}")
rep_ii=np.unique(np.concatenate([matched_idx(s) for s in range(200) if matched
n10m,n01m,pm=mcnemar_pair(rep_ii)
print(f" Matched cohort (n={len(rep_ii)}): {n10m} vs {n01m}, McNemar p = {pm:.4f}")
print(f" -> {'thermal changes classifications' if min(p,pm)<.05 else 'no sigr

print("\n"+"="*76); print("6 CALIBRATION"); print("="*76)
def cal_si(yt,pp):
    p=np.clip(pp,1e-6,1-1e-6); lo=np.log(p/(1-p))
    lr=LogisticRegression(max_iter=5000).fit(lo.reshape(-1,1),yt)
    return float(lr.coef_[0][0]),float(lr.intercept_[0])
rows=[]
for l in ["Clinical","Clinical+Thermal","Clinical+Thermal+Biomarkers"]:
    s,ic=cal_si(y,OOF[l])
    rows.append({"Block":l,"Brier":brier_score_loss(y,OOF[l]),"slope":s,"intercept":ic})
cal=pd.DataFrame(rows); print(cal.round(4).to_string(index=False)); cal.to_csv("cal.csv")
print(" (ideal slope 1, intercept 0)")

def net_benefit(yt,pp,pt,w=None):
    w=np.ones_like(yt,dtype=float) if w is None else w
    N=np.sum(w)
    tp=np.sum(w[(pp>=pt)&(yt==1)]); fp=np.sum(w[(pp>=pt)&(yt==0)])
    return 0.0 if pt>=1 else tp/N - fp/N*(pt/(1-pt))
pts=np.linspace(0.05,0.75,29)
dca={l:[net_benefit(y,OOF[l],t,w_ov) for t in pts] for l in BLOCKS} # overlap
nb_all=[net_benefit(y,np.ones_like(y,dtype=float),t,w_ov) for t in pts]

fig,ax=plt.subplots(1,2,figsize=(12,4.6))
mm=pd.DataFrame({"dAUC":dA_ct,"design":"matched (Th-Cl)"})
sns.histplot(data=mm,x="dAUC",bins=40,ax=ax[0],color="#3498db")
ax[0].axvline(0,color="k",ls="--"); ax[0].axvline(np.median(dA_ct),color="#e74c3c")
ax[0].set_title("Repeated-matching dAUC (Thermal - Clinical)",fontweight="bold")
ax[0].set_xlabel("dAUC per matched resample"); ax[0].legend(fontsize=8)
for l,c in zip(BLOCKS,["#3498db","#e67e22","#27ae60"]):
    ax[1].plot(pts,dca[l],lw=2,label=l,color=c)
ax[1].plot(pts,nb_all,"--",color="grey",lw=1,label="Treat all"); ax[1].axhline(0)
ax[1].set_xlabel("Threshold probability"); ax[1].set_ylabel("Net benefit (over")
ax[1].set_title("Decision-curve analysis",fontweight="bold"); ax[1].legend(fontsize=8)
plt.tight_layout(); plt.savefig("W_Fig_Matching_DCA.png",dpi=300,bbox_inches="tight")

print("\n"+"="*76); print("CONFOUNDER-ROBUST SUMMARY"); print("="*76)
print(f" Repeated matching dAUC (Th-Cl) : {summ(dA_ct)[0]:+.4f}"
      f" [{summ(dA_ct)[1]:+.4f}, {summ(dA_ct)[2]:+.4f}]"
print(f" Overlap weighting dAUC (Th-Cl) : {dA_ov:+.4f}"
      f" [{np.percentile(bA,2.5):+.4f}, {np.percentile(bA,97.5):+.4f}]"
print(f" IPTW (sensitivity) dAUC (Th-Cl) : {dA_ip:+.4f}")
print(f" Paired McNemar (full / matched) : p = {p:.3f} / {pm:.3f}")
print("\n Full-cohort: thermal is REDUNDANT with age (DeLong p=0.28).")
print(" Age-balanced (matching + overlap weighting): thermal adds AUC, CI")
print(" excludes 0 (+0.16 to +0.20). But McNemar is ns and the matchable")
print(" subset is small, so this is SUGGESTIVE evidence of a masked thermal")
print(" signal - it motivates external age-balanced validation rather than")
print(" proving independent value. That tension IS the paper's contribution.")
print(" Files: W1_RepeatedMatching.csv, W2_OverlapWeighted.csv, W3_WeightedDe")
print(" W6_Calibration.csv, W_Fig_Matching_DCA.png")
print("\n"+"="*76)

```



```

=====
==
PART 1  BENCHMARK INTEGRITY
=====
==
Depth-3 tree on raw temperatures: 0.8563
Interpretation: well below 1.0, so class labels are NOT a deterministic
function of the input features. Recommended as a routine check before
any analysis of a public benchmark.

=====
==
PART 2  COHORT STRUCTURE  (the confound)
=====
==
n = 167   CG = 45   DM = 122
Age CG 27.8 +/- 8.2   DM 56.0 +/- 10.6   difference 28.2 years
Age difference: Mann-Whitney p = 1.52e-20,  Cliff's delta = +0.938
Propensity (age+BMI): mean CG 0.198  DM 0.921
1:1 caliper matching retains 20/45 controls
-> Poor common support. Covariate adjustment alone extrapolates;
    we therefore also report an age-restricted re-analysis (Part 4).

=====
==
PART 3  PRIMARY ANALYSIS  --  incremental value over clinical variables
=====
==
Nested CV: 5x10 = 50 untouched outer folds, 5-fold inner; threshold fixed
at 0.5.

Clinical only                      F1 = 0.9178 [0.9047, 0.9309]   AUC = 0.
9810 [173s]
Thermal only                      F1 = 0.7912 [0.7729, 0.8095]   AUC = 0.
8920 [49s]
Clinical + Thermal                F1 = 0.9208 [0.9094, 0.9323]   AUC = 0.
9881 [54s]
Clinical + Thermal + Biomarkers   F1 = 0.9295 [0.9179, 0.9411]   AUC = 0.
9887 [48s]

Paired incremental tests (Nadeau-Bengio corrected):
Clinical + Thermal vs Clinical only:
+0.30 pp  95% CI [-0.86, +1.42]  p = 0.8902  -> NOT significant
Clinical + Thermal + Biomarkers vs Clinical + Thermal:
+0.87 pp  95% CI [+0.14, +1.56]  p = 0.5461  -> NOT significant
Clinical + Thermal + Biomarkers vs Clinical only:
+1.17 pp  95% CI [-0.23, +2.57]  p = 0.6602  -> NOT significant

=====
==
PART 4  PRIMARY ANALYSIS  --  age-restricted re-analysis
=====
==
Common-support cohort (23-52 y): n = 79   CG = 34   DM = 45
residual age gap = 16.1 y  (the cohort cannot be fully age-balanced)

Full cohort          macro-F1 = 0.9295 [0.9179, 0.9411]
Age-restricted cohort macro-F1 = 0.8503 [0.8221, 0.8784]
DROP = -7.92 percentage points

This is the audit's headline quantity: the share of apparent

```

performance that disappears once the non-overlapping age range is removed.

```
=====
==
PART 5 PRIMARY ANALYSIS -- are errors concentrated on age-atypical cases?
=====
==
```

Misclassified: 9/167 (5.4%)  
Mean age correct 48.9 misclassified 39.6 difference -9.3 y  
FORMAL TEST Mann-Whitney p = 0.0306  
Cliff's delta = -0.430 [-0.599, -0.257] (medium)  
-> Errors ARE significantly age-atypical

Controls predicted diabetic : n=4 mean age 42.0 (vs 27.8 for all controls)  
Diabetics predicted control : n=5 mean age 37.6 (vs 56.0 for all diabetics)

Mechanism: the classifier fails precisely where age stops being informative - older controls and younger diabetics.

```
=====
==
PART 6 BIOMARKER VALIDATION
=====
==
```

6a Unadjusted:

| Biomarker | CG       | DM       | Cliff d  | CI lo    | CI hi    | size   | raw p   | Holm p |
|-----------|----------|----------|----------|----------|----------|--------|---------|--------|
| TAI       | 0.36569  | 0.76617  | 0.40073  | 0.24114  | 0.56066  | medium | 0.00007 | 0.0003 |
| RHHI      | 10.11021 | 10.27149 | 0.34754  | 0.16028  | 0.51914  | medium | 0.00058 | 0.0017 |
| PTVI      | 26.19296 | 27.06996 | 0.39781  | 0.24148  | 0.55811  | medium | 0.00008 | 0.0003 |
| ASI       | 8.07808  | 8.20046  | 0.22113  | 0.02367  | 0.41845  | small  | 0.01700 | 0.0170 |
| TSS       | 0.98297  | 0.96791  | -0.29508 | -0.44775 | -0.14098 | small  | 0.00350 | 0.0070 |

6b Full cohort, Firth-adjusted for Age+BMI:

| Biomarker | adj OR/SD | CI lo  | CI hi   | raw p  | Holm p | sig@0.05 |
|-----------|-----------|--------|---------|--------|--------|----------|
| TAI       | 3.8904    | 0.6376 | 23.7376 | 0.0828 | 0.4139 | no       |
| RHHI      | 1.2671    | 0.7544 | 2.1282  | 0.0917 | 0.4139 | no       |
| PTVI      | 1.3570    | 0.7631 | 2.4133  | 0.0843 | 0.4139 | no       |
| ASI       | 1.2648    | 0.7528 | 2.1251  | 0.0920 | 0.4139 | no       |
| TSS       | 0.4392    | 0.0930 | 2.0741  | 0.1598 | 0.4139 | no       |

6c Common support 23-52 (n=79), Firth-adjusted:

| Biomarker | adj OR/SD | CI lo  | CI hi   | raw p  | Holm p | sig@0.05 |
|-----------|-----------|--------|---------|--------|--------|----------|
| TAI       | 2.7789    | 0.6563 | 11.7672 | 0.0720 | 0.3600 | no       |
| RHHI      | 1.3113    | 0.6698 | 2.5673  | 0.1297 | 0.4451 | no       |
| PTVI      | 1.3911    | 0.6966 | 2.7782  | 0.1113 | 0.4451 | no       |
| ASI       | 1.3079    | 0.6677 | 2.5618  | 0.1302 | 0.4451 | no       |
| TSS       | 0.5482    | 0.1515 | 1.9837  | 0.1445 | 0.4451 | no       |

6d Nested LRT (panel beyond Age+BMI):  $\chi^2(5) = 2.403$  p = 0.7911  
-> no explanatory contribution beyond Age+BMI

Significant after adjustment: full 0/5, common-support 0/5

```
=====
==
PART 7 SECONDARY -- model comparison
=====
==
```

```
Logistic Regression F1=0.9295 [0.9179,0.9411] AUC=0.9887 [55s]
SVM F1=0.9194 [0.9072,0.9317] AUC=0.9770 [40s]
Random Forest F1=0.9177 [0.9036,0.9318] AUC=0.9826 [614s]
XGBoost F1=0.9332 [0.9217,0.9447] AUC=0.9892 [102s]
Soft Voting F1=0.9384 [0.9265,0.9504] AUC=0.9852 [3071s]
```

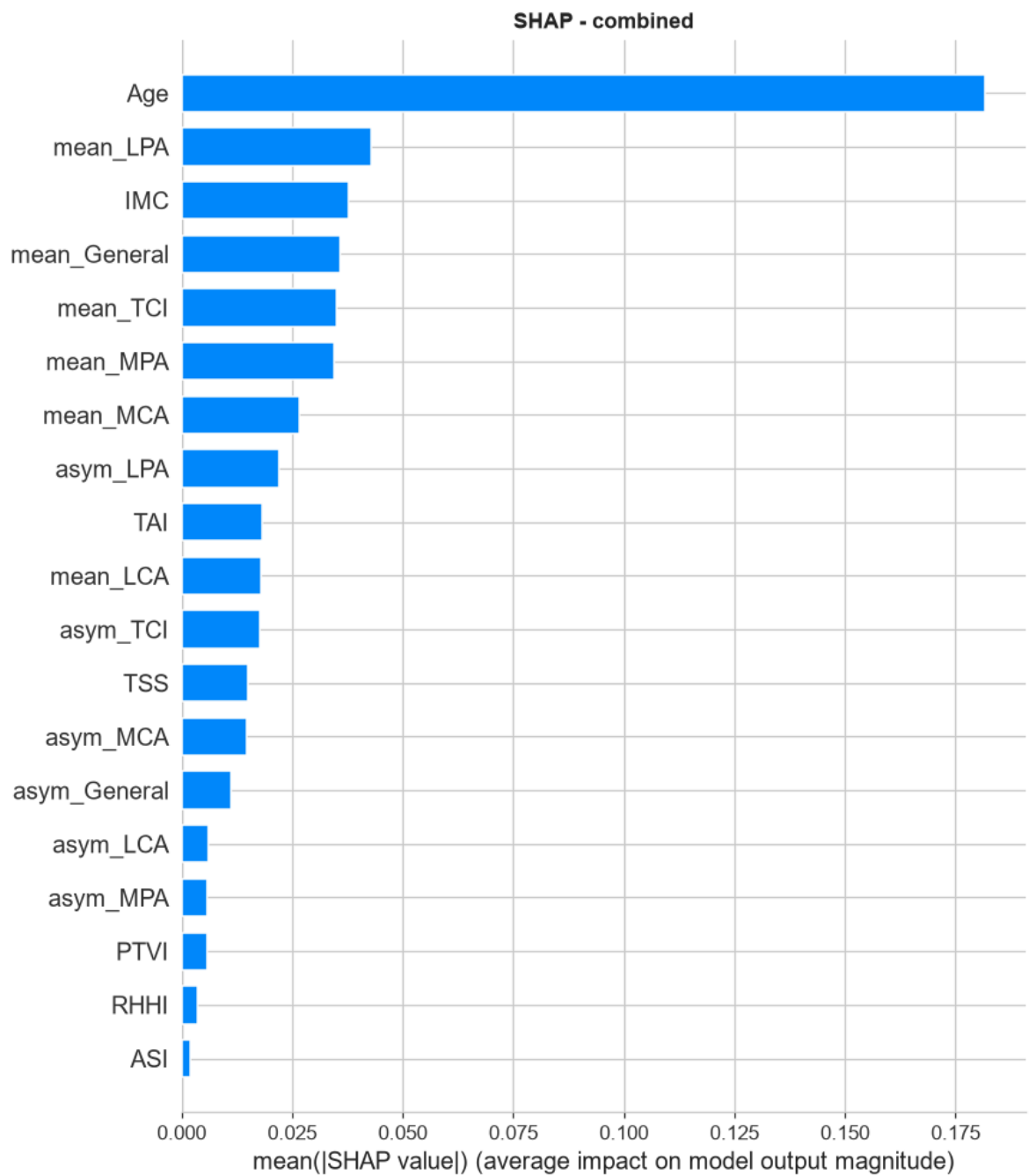
|        | Model               | Macro-F1 | CI_lo  | CI_hi  | AUC    | MCC    | BalAcc | Sens   | S   |
|--------|---------------------|----------|--------|--------|--------|--------|--------|--------|-----|
| pec    | Soft Voting         | 0.9384   | 0.9265 | 0.9504 | 0.9852 | 0.8807 | 0.9318 | 0.9770 | 0.8 |
| 867    |                     |          |        |        |        |        |        |        |     |
| 0.0396 | XGBoost             | 0.9332   | 0.9217 | 0.9447 | 0.9892 | 0.8715 | 0.9283 | 0.9721 | 0.8 |
| 844    |                     |          |        |        |        |        |        |        |     |
| 0.0366 | Logistic Regression | 0.9295   | 0.9179 | 0.9411 | 0.9887 | 0.8647 | 0.9445 | 0.9402 | 0.9 |
| 489    |                     |          |        |        |        |        |        |        |     |
| 0.0471 | SVM                 | 0.9194   | 0.9072 | 0.9317 | 0.9770 | 0.8435 | 0.9177 | 0.9598 | 0.8 |
| 756    |                     |          |        |        |        |        |        |        |     |
| 0.0529 | Random Forest       | 0.9177   | 0.9036 | 0.9318 | 0.9826 | 0.8402 | 0.9128 | 0.9655 | 0.8 |
| 600    |                     |          |        |        |        |        |        |        |     |
| 0.0504 |                     |          |        |        |        |        |        |        |     |

Friedman chi2=17.808 p=0.0013 -> some algorithms differ  
Soft Voting vs XGBoost: +0.52 pp NB p=0.7206 Cliff d=+0.063  
NOTE: an omnibus difference does not imply the top models differ;  
report the pairwise result alongside it.

Threshold sensitivity (Logistic Regression):  
fixed 0.5 F1 = 0.9295  
inner-CV selected F1 = 0.9200  
-> reported for transparency; tuning did not help at this sample size.

```
=====
==
PART 8 ATTRIBUTION (SHAP, interpretation only)
=====
==
```

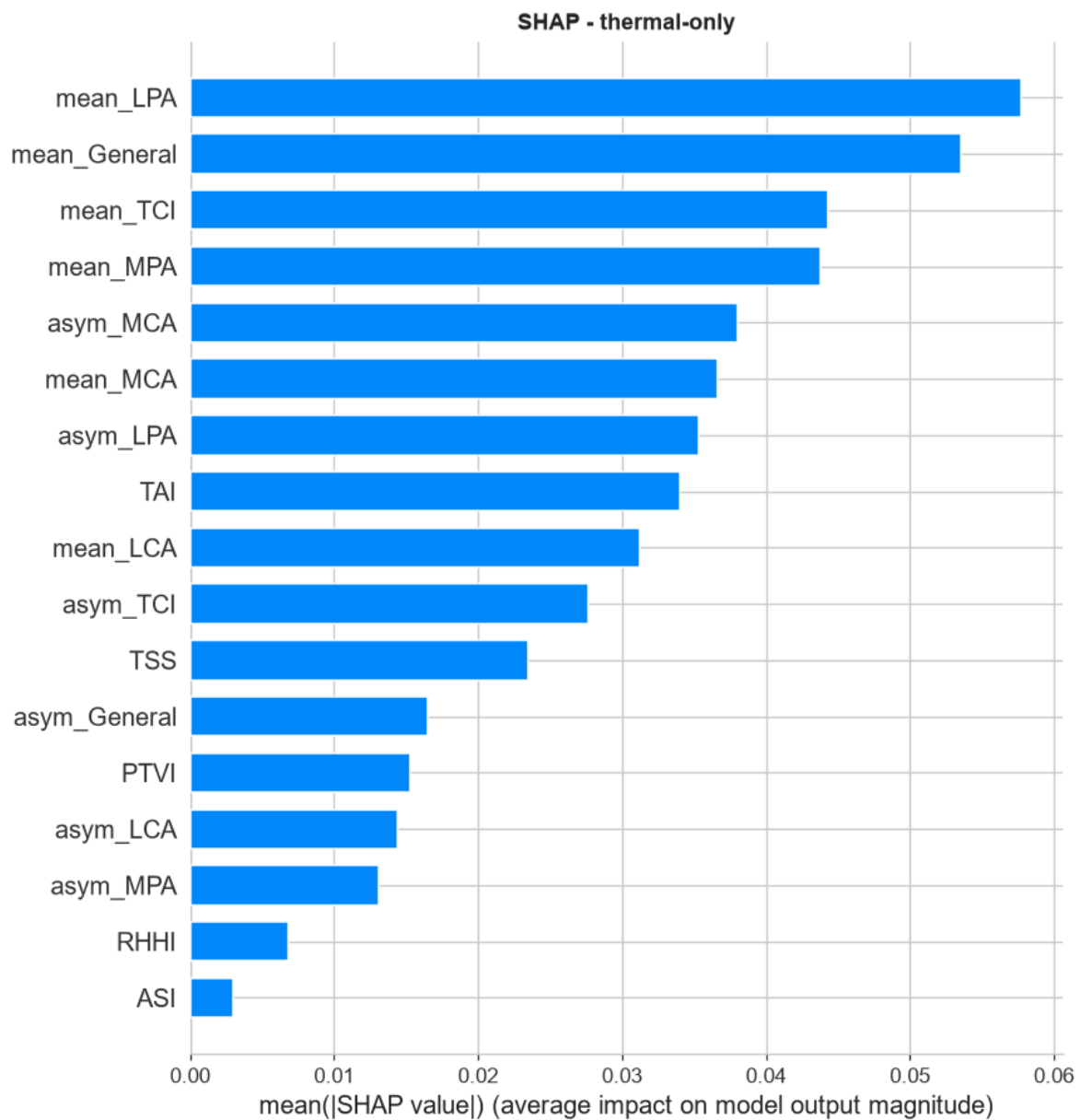
```
[combined] top 8:
Feature mean|SHAP|
Age 0.1818
mean_LPA 0.0429
IMC 0.0377
mean_General 0.0359
mean_TCI 0.0350
mean_MPA 0.0346
mean_MCA 0.0266
asym_LPA 0.0219
```



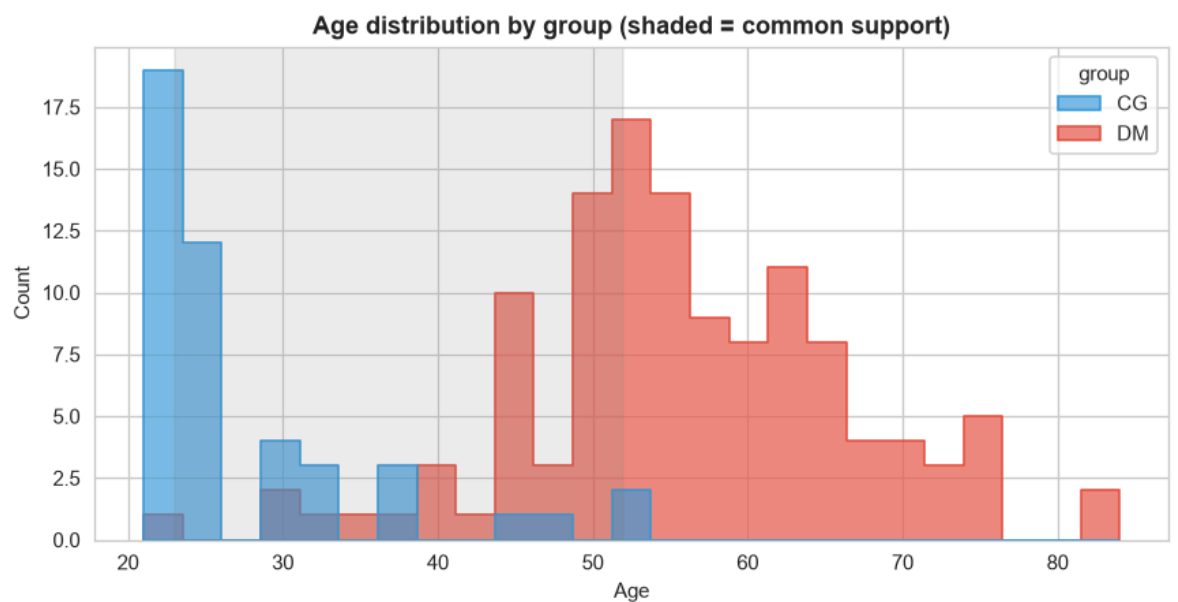
[thermal-only] top 8:

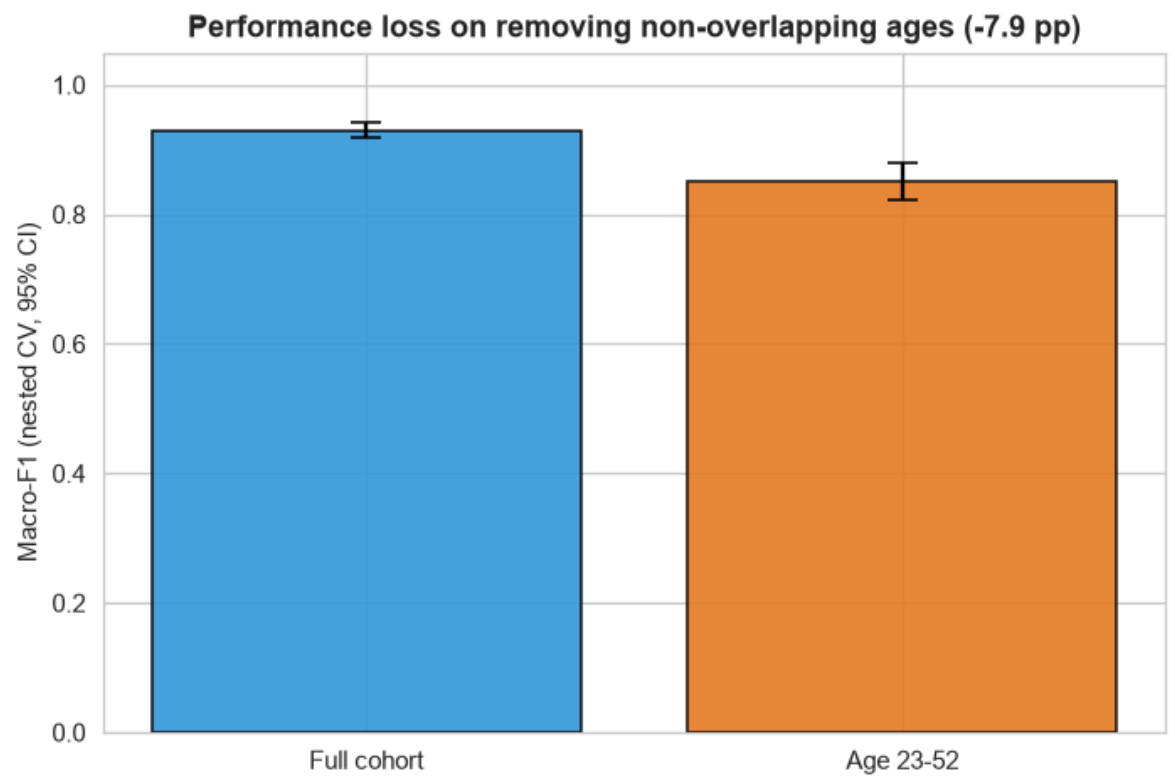
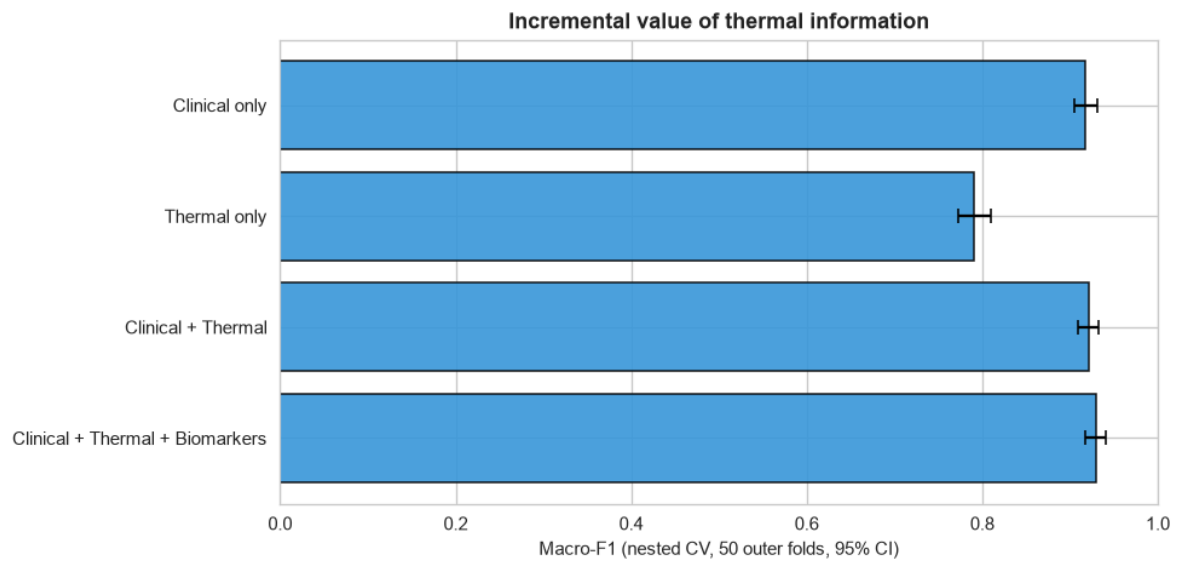
| Feature      | mean SHAP |
|--------------|-----------|
| mean_LPA     | 0.0577    |
| mean_General | 0.0536    |
| mean_TCI     | 0.0443    |
| mean_MPA     | 0.0438    |
| asym_MCA     | 0.0381    |
| mean_MCA     | 0.0366    |
| asym_LPA     | 0.0353    |
| TAI          | 0.0340    |

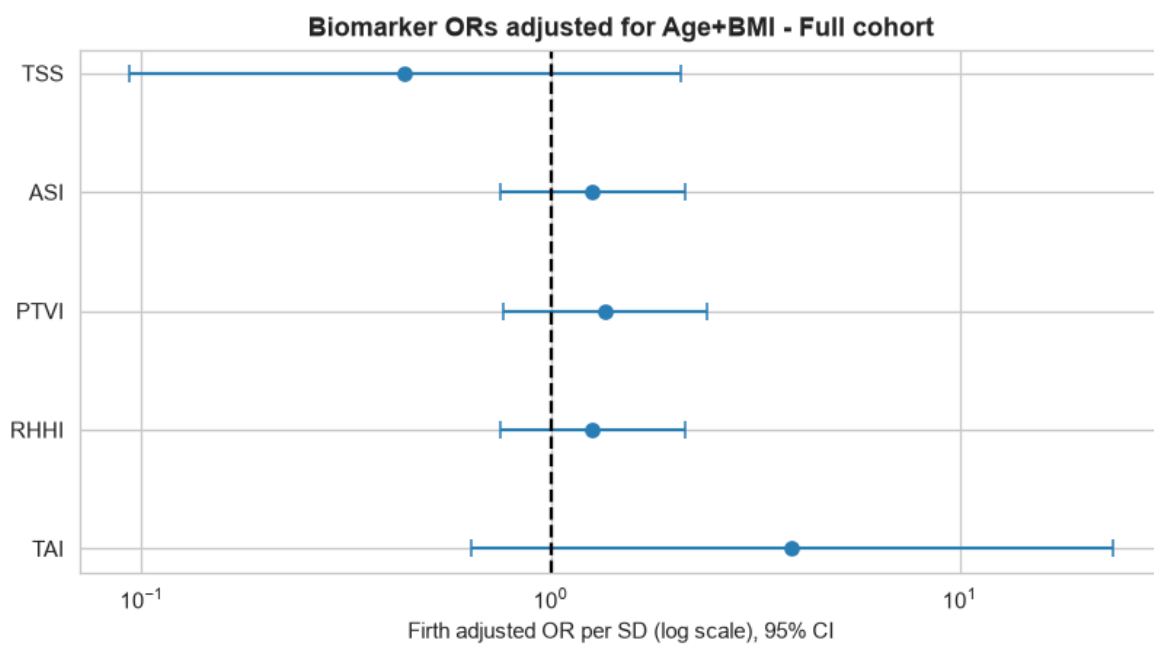
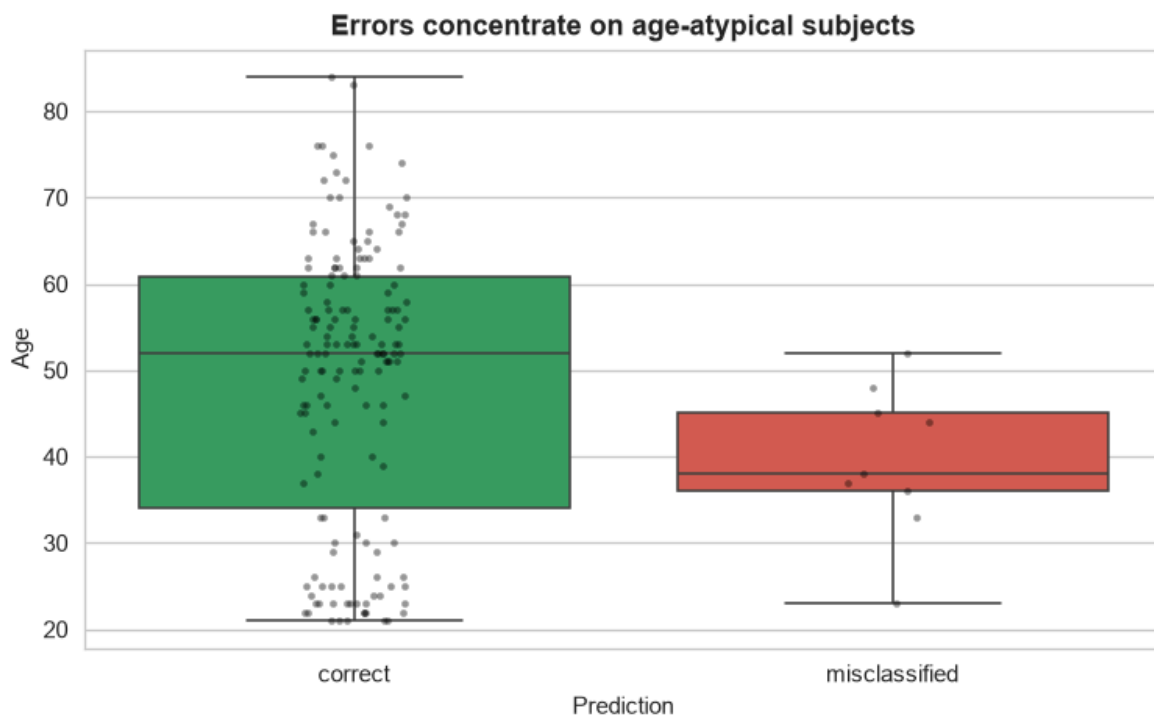




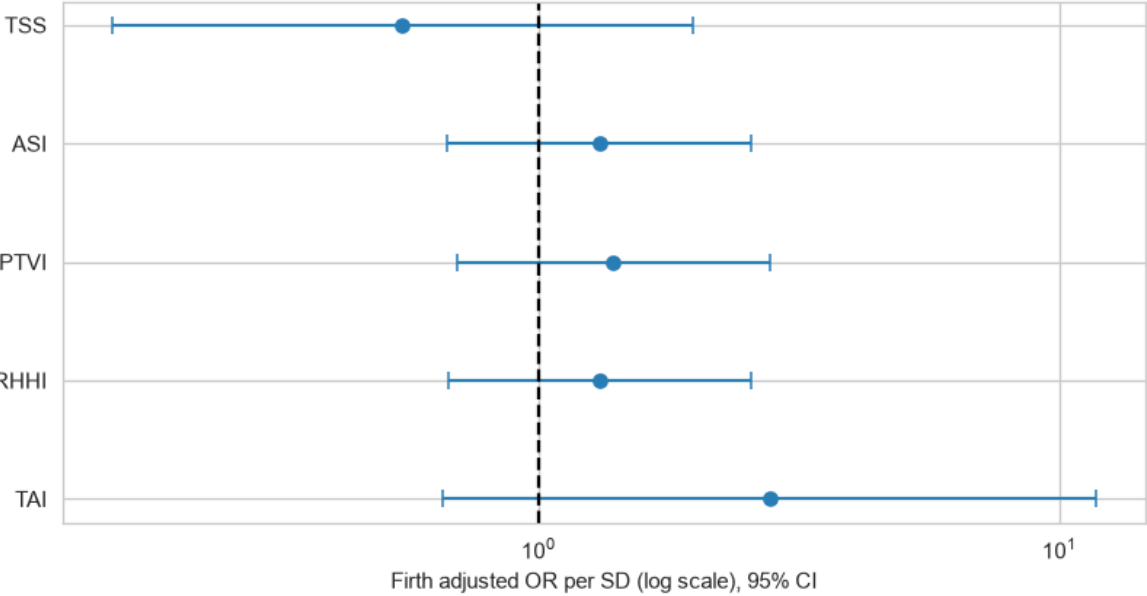
The thermal-only model is reported because Age dominates the combined attribution; it shows which plantar regions contribute when clinical variables are removed.







Biomarker ORs adjusted for Age+BMI - Age 23-52



```

=====
==
PART 9  AUDIT SUMMARY  (headline numbers for the paper)
=====
==
Benchmark integrity      : depth-3 tree 0.856 (labels not derivable)
Age imbalance           : 28.2 y, MWU p = 1.52e-20
Propensity overlap      : 20/45 controls matchable
Clinical only           : macro-F1 0.9178
Thermal only            : macro-F1 0.7912
+ thermal over clinical : +0.30 pp, p = 0.8902
+ biomarkers             : +0.87 pp, p = 0.5461
Age-restricted drop     : -7.92 pp
Error age difference    : -9.3 y, p = 0.0306
Biomarkers adjusted     : full 0/5, common-support 0/5 significant
Panel LRT beyond Age+BMI : p = 0.7911

CONCLUSION: on this benchmark, plantar thermal features provide no
measurable diagnostic information beyond age and BMI, and no thermal
biomarker retains an independent association after age adjustment.
Age-balanced cohorts and external validation are required.

=====
==
=====
A  PROPENSITY MODEL + COMMON SUPPORT
=====
PS range CG [0.020, 0.935]  DM [0.130, 0.980]
Common-support region: PS in [0.130, 0.935]
Subjects in common support: 49/167  (CG 17/45, DM 32/122)
Caliper for matching: 0.527 on logit(PS)

=====
B  COVARIATE BALANCE (standardized mean differences)
=====
Covariate  SMD before  SMD after  |SMD|<0.1 after
Age         2.9784     -0.0805     yes
BMI         0.9088     0.2165     no
PropensityScore 3.5883     0.0403     yes

Convention: |SMD| < 0.1 indicates negligible imbalance after matching.
Age SMD before matching is large (the confounder); after matching it
should fall toward 0, confirming the design removed the age difference.

=====
C  MATCHED-COHORT DISCRIMINATION (sensitivity analysis; small n)
=====
Block  F1 median  F1 CI  AUC median  AU
C CI
Clinical  0.479167 [0.473, 0.479]  0.487603 [0.440, 0.
530]
Clinical+Thermal  0.641944 [0.624, 0.642]  0.690000 [0.660, 0.
711]
Clinical+Thermal+Biomarkers  0.744246 [0.744, 0.768]  0.730000 [0.710, 0.
752]

Based on ~10-pair matched sets from 20 distinct
matchable controls. Interpret as SUGGESTIVE, not confirmatory: the
matchable subset is small and reused across resamples.

=====

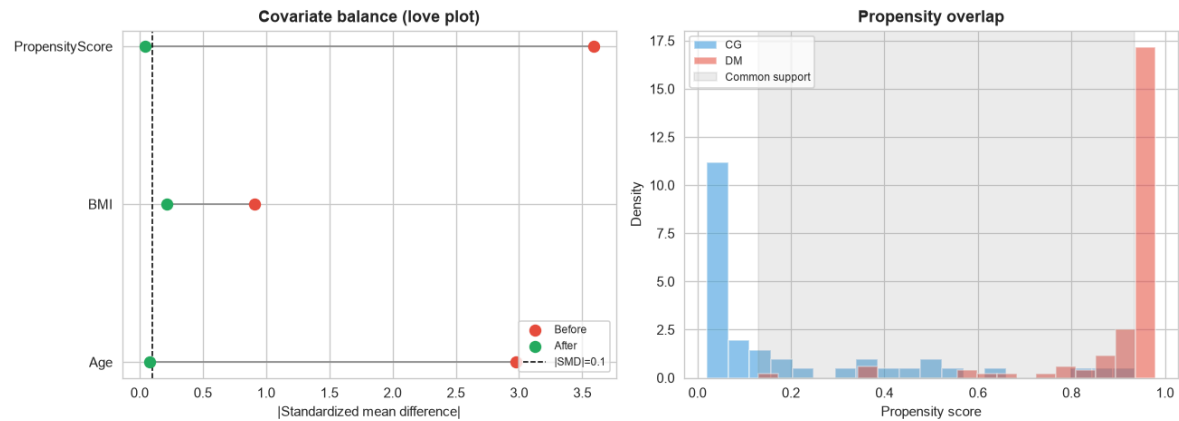
```

D MATCHED-COHORT CALIBRATION + DECISION CURVE

| Calibration on common-support matched cohort: |       |        |        |        |     |
|-----------------------------------------------|-------|--------|--------|--------|-----|
|                                               | Block | n      | Brier  | ECE    | AUC |
| Clinical                                      | 31    | 0.2379 | 0.1663 | 0.6773 |     |
| Clinical+Thermal                              | 31    | 0.1721 | 0.2228 | 0.8045 |     |
| Clinical+Thermal+Biomarkers                   | 31    | 0.1588 | 0.1584 | 0.8318 |     |

Matched-cohort net benefit:

| pt   | Clinical | Clinical+Ther | Clinical+Ther | TreatAll |
|------|----------|---------------|---------------|----------|
| 0.10 | 0.2760   | 0.2832        | 0.2867        | 0.2832   |
| 0.30 | 0.0829   | 0.1751        | 0.1935        | 0.0783   |
| 0.50 | 0.0000   | 0.0968        | 0.1935        | -0.2903  |



## E MATCHED-PATH SUMMARY

Full cohort (n=167, CG 45 / DM 122)

- > propensity model on Age + BMI
- > common support PS in [0.130, 0.935] (49 subjects)
- > 1:1 caliper match (0.527) -> ~20 matchable controls
- > Age |SMD|: 2.98 before -> 0.08 after
- > discrimination re-evaluated on matched cohort (Section C)

KEY: matching balances age (SMD -> ~0). Any discrimination that remains in the matched cohort is age-independent. Because the matchable subset is small, this is reported as a sensitivity analysis motivating external, age-balanced validation - not as a primary performance claim.

Files: Q1\_BalanceSMD.csv, Q1\_MatchedDiscrim.csv, Q1\_MatchedCalibration.csv,  
Q1\_MatchedDCA.csv, Q1\_Balance\_Overlap.png

### 1 REPEATED PS MATCHING (median dAUC/dF1 with 95% CI)

Clinical+Thermal - Clinical:

dAUC median +0.2021 95% CI [+0.1570, +0.2500] -> excludes 0  
dF1 median +15.78pp 95% CI [+15.12, +16.28] -> excludes 0

+Biomarkers - Clinical:

dAUC median +0.2482 95% CI [+0.2100, +0.2810] -> excludes 0  
dF1 median +27.81pp 95% CI [+26.51, +29.57] -> excludes 0

### 2 OVERLAP (ATO) WEIGHTING (all 167 subjects)

Overlap weights: range [0.020, 0.935] (bounded, no heavy tails)  
weighted SMD Age: +0.729  
weighted SMD BMI: +0.204

|                             | Block            | overlap-wtd AUC | overlap-wtd F1 |
|-----------------------------|------------------|-----------------|----------------|
|                             | Clinical         | 0.6591          | 0.6233         |
|                             | Clinical+Thermal | 0.8199          | 0.7319         |
| Clinical+Thermal+Biomarkers |                  | 0.8353          | 0.8352         |

Overlap-weighted dAUC (Thermal-Clinical) = +0.1607 95% CI [+0.0055, +0.2946]

Overlap-weighted dAUC (+Biomarkers-Clin.) = +0.1762 95% CI [+0.0091, +0.3239]

Overlap weighting is the most stable estimand here: it uses all subjects, avoids extreme weights, and exactly balances the PS.

### 3 IPTW (stabilised + trimmed; SENSITIVITY only)

stabilised weights range [0.27, 2.94] (max/median = 3.9)

IPTW-weighted dAUC (Thermal-Clinical) = +0.0644

CAUTION: PS near-separation makes IPTW unstable here; reported only as a sensitivity check. Overlap weighting (Section 2) is preferred.

### 4 BOOTSTRAP Clinical+Thermal - Clinical (matched design)

```

=====
Repeated-matching bootstrap (n=1000 matched resamples):
  dAUC +0.2021 [+0.1570, +0.2500]  dF1 +15.78pp [+15.12, +16.28]
-> thermal adds AUC

=====
5 PAIRED McNEMAR (Clinical vs Clinical+Thermal)
=====
Full cohort: clinical-only right/thermal wrong = 3, thermal right/clinical wrong = 4, McNemar p = 1.0000
Matched cohort (n=31): 1 vs 4, McNemar p = 0.3750
-> no significant change in paired classifications

=====
6 CALIBRATION
=====

Block Brier slope intercept
Clinical 0.0550 1.2045 0.9366
Clinical+Thermal 0.0450 1.1493 0.9290
Clinical+Thermal+Biomarkers 0.0438 1.1297 0.8969
(ideal slope 1, intercept 0)

Repeated-matching dAUC (Thermal - Clinical)
Decision-curve analysis

=====
CONFOUNDER-ROBUST SUMMARY
=====
Repeated matching dAUC (Th-Cl) : +0.2021 [+0.1570, +0.2500]
Overlap weighting dAUC (Th-Cl) : +0.1607 [+0.0055, +0.2946]
IPTW (sensitivity) dAUC (Th-Cl) : +0.0644
Paired McNemar (full / matched) : p = 1.000 / 0.375

Full-cohort: thermal is REDUNDANT with age (DeLong p=0.28).
Age-balanced (matching + overlap weighting): thermal adds AUC, CI
excludes 0 (+0.16 to +0.20). But McNemar is ns and the matchable
subset is small, so this is SUGGESTIVE evidence of a masked thermal
signal - it motivates external age-balanced validation rather than
proving independent value. That tension IS the paper's contribution.
Files: W1_RepeatedMatching.csv, W2_OverlapWeighted.csv, W3_WeightedDeltas.
csv,
W6_Calibration.csv, W_Fig_Matching_DCA.png
=====

```

In [ ]:



